20.4 Slack 通道（Bolt）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~650k tokens，输出 ~60k tokens（本章合计）

Slack 是企业级协作平台的标杆，也是 OpenClaw 支持的最"重量级"通道之一——它的权限模型、事件体系和交互组件远比其他即时通讯平台复杂。OpenClaw 的 Slack 模块位于 `src/slack/`，代码量达到 65 个文件，是所有通道中最庞大的实现。本节将从 Bolt SDK 对接、双 Token 架构到线程会话管理三个维度，深入分析 Slack 通道的核心实现。

20.4.1 Slack Bolt SDK 对接

Bolt 框架简介

> **衍生解释**：Slack Bolt 是 Slack 官方提供的应用开发框架（有 JavaScript、Python、Java 三个版本）。它封装了 Slack Events API、Web API 和 Socket Mode 的底层通信细节，提供事件监听、命令处理、交互响应等高层 API。Bolt 对于 Slack 应用开发，类似于 Express 对于 HTTP 服务开发——它并非必须，但极大地简化了样板代码。

OpenClaw 使用 `@slack/bolt`（JavaScript 版）作为 Slack 集成的核心依赖。所有 Slack 消息的收发、事件处理和斜杠命令都构建在 Bolt 之上。

monitorSlackProvider：启动入口

Slack 通道的入口函数是 `monitorSlackProvider()`，位于 `src/slack/monitor/provider.ts`，它是整个 Slack 监听循环的编排中心：

```typescript
// src/slack/monitor/provider.ts（简化）
import SlackBolt from "@slack/bolt";

// 处理 CJS/ESM 兼容问题：Bun 可以直接具名导入，Node ESM 不行
const slackBolt = slackBoltModule.App
  ? slackBoltModule : slackBoltModule.default ?? slackBoltModule;
const { App, HTTPReceiver } = slackBolt;

export async function monitorSlackProvider(opts: MonitorSlackOpts = {}) {
  const cfg = opts.config ?? loadConfig();
  const account = resolveSlackAccount({ cfg, accountId: opts.accountId });

  // 1. 解析配置：Token、会话范围、DM 策略、群组策略...
  const botToken = resolveSlackBotToken(opts.botToken ?? account.botToken);
  const appToken = resolveSlackAppToken(opts.appToken ?? account.appToken);
  const sessionScope: SessionScope = sessionCfg?.scope ?? "per-sender";

  // 2. 根据模式创建 Bolt App
  const app = new App(
    slackMode === "socket"
      ? { token: botToken, appToken, socketMode: true, clientOptions }
      : { token: botToken, receiver: receiver ?? undefined, clientOptions }
  );

  // 3. 身份验证
  const auth = await app.client.auth.test({ token: botToken });
  botUserId = auth.user_id ?? "";
  teamId = auth.team_id ?? "";

  // 4. 构建上下文 → 注册事件 → 注册斜杠命令 → 启动
  const ctx = createSlackMonitorContext({ ... });
  const handleSlackMessage = createSlackMessageHandler({ ctx, account });
  registerSlackMonitorEvents({ ctx, account, handleSlackMessage });
  registerSlackMonitorSlashCommands({ ctx, account });

  // 5. 启动连接
  if (slackMode === "socket") {
    await app.start();
    runtime.log?.("slack socket mode connected");
  }
}
```

整个启动流程可以归纳为以下步骤：

步骤	内容	关键函数
1	加载并合并配置	`resolveSlackAccount()`
2	创建 Bolt App 实例	`new App()`
3	调用 `auth.test` 获取 Bot 身份	`app.client.auth.test()`
4	构建监听上下文	`createSlackMonitorContext()`
5	创建消息处理器	`createSlackMessageHandler()`
6	注册所有事件监听	`registerSlackMonitorEvents()`
7	注册斜杠命令	`registerSlackMonitorSlashCommands()`
8	异步解析频道/用户白名单	`resolveSlackChannelAllowlist()`
9	启动 Socket Mode / HTTP 监听	`app.start()`

两种连接模式

Slack 提供两种接收事件的方式，OpenClaw 都支持：

Socket Mode（默认）：通过 WebSocket 长连接接收事件。适合开发环境和无公网 IP 的部署场景。需要 App Token。

HTTP Mode：Slack 将事件推送到指定 URL。适合生产环境，需要公网可达的 HTTPS 端点和 Signing Secret。

```
// 根据配置选择接收器
const receiver = slackMode === "http"
  ? new HTTPReceiver({
      signingSecret: signingSecret ?? "",
      endpoints: slackWebhookPath,
    })
  : null;
```

HTTP 模式下，OpenClaw 还注册了一个 HTTP Handler 到 Gateway 服务器的路由系统中：

```
if (slackMode === "http" && slackHttpHandler) {
  unregisterHttpHandler = registerSlackHttpHandler({
    path: slackWebhookPath,
    handler: slackHttpHandler,
    log: runtime.log,
    accountId: account.accountId,
  });
}
```

事件注册架构

事件注册被分解为五个独立模块，每个负责一类 Slack 事件：

```
// src/slack/monitor/events.ts
export function registerSlackMonitorEvents(params) {
  registerSlackMessageEvents({ ctx, handleSlackMessage });  // message + app_mention
  registerSlackReactionEvents({ ctx });                      // reaction_added/removed
  registerSlackMemberEvents({ ctx });                        // member_joined/left_chan
  registerSlackChannelEvents({ ctx });                       // channel_created/renamed
  registerSlackPinEvents({ ctx });                           // pin_added/removed
}
```

其中最核心的是 `registerSlackMessageEvents`，它同时监听 `message` 和 `app_mention` 两种事件：

```
// src/slack/monitor/events/messages.ts
ctx.app.event("message", async ({ event, body }) => {
  if (ctx.shouldDropMismatchedSlackEvent(body)) return;
  const message = event as SlackMessageEvent;

  // 处理编辑、删除、线程广播等子类型
  if (message.subtype === "message_changed") { /* 系统事件 */ }
  if (message.subtype === "message_deleted") { /* 系统事件 */ }
  if (message.subtype === "thread_broadcast") { /* 系统事件 */ }

  // 常规消息交给统一处理器
  await handleSlackMessage(message, { source: "message" });
});

ctx.app.event("app_mention", async ({ event, body }) => {
  // @mention 事件直接标记为 wasMentioned: true
  await handleSlackMessage(mention as SlackMessageEvent, {
    source: "app_mention",
    wasMentioned: true,
  });
});
```

> **衍生解释**：Slack 中 `message` 事件覆盖了几乎所有消息相关的场景（新消息、编辑、删除、文件分享等），通过 `subtype` 字段区分。而 `app_mention` 是一个独立事件，仅在用户 `@` 你的 Bot 时触发。OpenClaw 同时监听两者，确保不遗漏任何需要响应的消息。

多事件去重

由于 `message` 和 `app_mention` 可能对同一条消息同时触发，OpenClaw 使用去重缓存避免重复处理：

```
// SlackMonitorContext 中的 markMessageSeen
const seenMessages = createDedupeCache({ ttlMs: 60_000, maxSize: 500 });

const markMessageSeen = (channelId: string | undefined, ts?: string) => {
  if (!channelId || !ts) return false;
  return seenMessages.check(`${channelId}:${ts}`);
};
```

`check()` 如果 key 已存在则返回 `true` （表示已见过），否则写入并返回 `false` 。TTL 60 秒足以覆盖 Slack 事件延迟。

api_app_id 校验

当多个 Slack 应用复用同一接收端时，OpenClaw 通过 `api_app_id` 和 `team_id` 过滤非自身事件：

```
const shouldDropMismatchedSlackEvent = (body: unknown) => {
  const raw = body as { api_app_id?: unknown; team_id?: unknown };
  if (params.apiAppId && incomingApiAppId && incomingApiAppId !== params.apiAppId) {
    return true;  // 不是本 App 的事件
  }
  if (params.teamId && incomingTeamId && incomingTeamId !== params.teamId) {
    return true;  // 不是本 Team 的事件
  }
  return false;
};
```

20.4.2 App Token + Bot Token 双 Token 架构

Slack 应用认证使用两套独立的 Token 体系，理解它们的区别是正确配置 Slack 通道的前提。

Token 类型对比

Token 类型	前缀	用途	示例场景
Bot Token	`xoxb-`	调用 Slack Web API（发消息、查信息等）	`chat.postMessage` 、 `users.info`
App Token	`xapp-`	建立 Socket Mode WebSocket 连接	实时接收事件，替代公网 Webhook
User Token	`xoxp-`	以用户身份调用 API（可选）	解析频道名称、获取用户列表

OpenClaw 要求 Bot Token 必须提供；App Token 在 Socket Mode 下必须提供，HTTP Mode 下不需要。

Token 解析链

Token 的来源遵循优先级链：函数参数 > 配置文件 > 环境变量。

```
// src/slack/accounts.ts
export function resolveSlackAccount(params) {
  const allowEnv = accountId === DEFAULT_ACCOUNT_ID;
  const envBot = allowEnv ? resolveSlackBotToken(process.env.SLACK_BOT_TOKEN) : undefin
  const envApp = allowEnv ? resolveSlackAppToken(process.env.SLACK_APP_TOKEN) : undefin
  const configBot = resolveSlackBotToken(merged.botToken);
  const configApp = resolveSlackAppToken(merged.appToken);

  // 配置优先于环境变量
  const botToken = configBot ?? envBot;
  const appToken = configApp ?? envApp;

  // 记录 Token 来源，方便调试
  const botTokenSource: SlackTokenSource = configBot ? "config" : envBot ? "env" : "non
}
```

> **设计亮点**： `botTokenSource` 字段记录 Token 来源（ `"config"` / `"env"` / `"none"` ），在发送失败时可以给出精确的错误信息（例如"设置 `channels.slack.accounts.default.botToken` 或环境变量 `SLACK_BOT_TOKEN` "）。

Token 一致性校验

App Token 中编码了 `api_app_id` ，OpenClaw 在启动时会从 App Token 中解析此 ID，并与 `auth.test` 返回的 `api_app_id` 比对：

```
function parseApiAppIdFromAppToken(raw?: string) {
  const match = /^xapp-\d-([a-z0-9]+)-/i.exec(token);
  return match?.[1]?.toUpperCase();
}

// 启动时校验
if (apiAppId && expectedApiAppIdFromAppToken && apiAppId !== expectedApiAppIdFromAppTok
  runtime.error?.(
    `slack token mismatch: bot token api_app_id=${apiAppId} ` +
    `but app token looks like api_app_id=${expectedApiAppIdFromAppToken}`
  );
}
```

这种校验可以在早期捕获"Bot Token 和 App Token 属于不同 Slack App"的配置错误。

多账户支持

与其他通道类似，Slack 也支持多账户配置。 `resolveSlackAccount()` 会合并全局 Slack 配置和特定账户配置：

```
function mergeSlackAccountConfig(cfg: OpenClawConfig, accountId: string): SlackAccountC
  const { accounts: _ignored, ...base } = cfg.channels?.slack ?? {};
  const account = resolveAccountConfig(cfg, accountId) ?? {};
  return { ...base, ...account };  // 账户级配置覆盖全局
}
```

配置结构示例：

```
channels:
  slack:
    groupPolicy: "open"           # 全局默认
    botToken: "xoxb-..."          # 全局 Bot Token
    accounts:
      customer-support:
        botToken: "xoxb-support-..."  # 覆盖 Bot Token
        groupPolicy: "allowlist"      # 覆盖群组策略
        channels:
          C0123456789:
            requireMention: true
```

WebClient 配置

所有 Slack API 调用通过 `@slack/web-api` 的 `WebClient` 进行，OpenClaw 为其配置了合理的重试策略：

```
// src/slack/client.ts
export const SLACK_DEFAULT_RETRY_OPTIONS: RetryOptions = {
  retries: 2,        // 最多重试 2 次
  factor: 2,         // 指数退避因子
  minTimeout: 500,   // 最短 500ms
  maxTimeout: 3000,  // 最长 3 秒
  randomize: true,   // 加入随机抖动
};

export function createSlackWebClient(token: string, options = {}) {
  return new WebClient(token, resolveSlackWebClientOptions(options));
}
```

20.4.3 线程（Thread）会话管理

Slack 的线程（Thread）是其最具特色的交互模式——在频道中的某条消息下可以开启子对话。OpenClaw 对线程的处理是整个 Slack 通道中最精巧的部分。

Slack 线程模型

> **衍生解释**：Slack 中每条消息都有一个 `ts` （时间戳 ID，如 `1716000000.123456` ）。如果一条消息是线程回复，它会携带 `thread_ts` 字段指向线程的根消息。因此， `thread_ts` 实际上是线程的唯一标识符。此外，如果消息回复了 Bot 发起的线程，它还会携带 `parent_user_id` 指向根消息的作者。

线程上下文解析

`resolveSlackThreadContext()` 是线程处理的核心函数，它从 Slack 消息中提取线程相关的所有信息：

```
// src/slack/threading.ts
export function resolveSlackThreadContext(params: {
  message: SlackMessageEvent | SlackAppMentionEvent;
  replyToMode: ReplyToMode;
}): SlackThreadContext {
  const incomingThreadTs = params.message.thread_ts;
  const messageTs = params.message.ts ?? eventTs;
  const hasThreadTs = typeof incomingThreadTs === "string" && incomingThreadTs.length >
  const isThreadReply =
    hasThreadTs && (incomingThreadTs !== messageTs || Boolean(params.message.parent_use

  // 线程 ID：如果是线程回复，用 thread_ts；否则根据 replyToMode 决定
  const messageThreadId = isThreadReply
    ? incomingThreadTs
    : params.replyToMode === "all"
      ? messageTs      // replyToMode=all：所有回复都开线程
      : undefined;     // replyToMode=off：不自动开线程

  return { incomingThreadTs, messageTs, isThreadReply, replyToId, messageThreadId };
}
```

缺失 thread_ts 的恢复

Slack 在某些边缘情况下会发送带有 `parent_user_id` 但没有 `thread_ts` 的消息（已知 Bug）。OpenClaw 的 `createSlackThreadTsResolver` 通过回查 `conversations.history` API 来恢复缺失的 `thread_ts`：

```typescript
// src/slack/monitor/thread-resolution.ts
export function createSlackThreadTsResolver(params) {
  const cache = new Map<string, ThreadTsCacheEntry>();
  const inflight = new Map<string, Promise<string | undefined>>();

  return {
    resolve: async (request) => {
      const { message } = request;
      // 只有 parent_user_id 存在但 thread_ts 缺失时才需要恢复
      if (!message.parent_user_id || message.thread_ts || !message.ts) {
        return message;
      }

      // 先查缓存
      const cached = getCached(cacheKey, now);
      if (cached !== undefined) {
        return cached ? { ...message, thread_ts: cached } : message;
      }

      // 去重正在进行的 API 请求（inflight dedup）
      let pending = inflight.get(cacheKey);
      if (!pending) {
        pending = resolveThreadTsFromHistory({
          client: params.client,
          channelId: message.channel,
          messageTs: message.ts,
        });
        inflight.set(cacheKey, pending);
      }

      const resolved = await pending;
      setCached(cacheKey, resolved ?? null, Date.now());
      return resolved ? { ...message, thread_ts: resolved } : message;
    }
  };
}
```

该设计有三个精巧之处：

1. **LRU 缓存**（60 秒 TTL，500 条上限）：避免对同一消息重复查询 API
2. **飞行中去重**（inflight dedup）：对同一消息的并发请求只执行一次 API 调用
3. **优雅降级**：API 调用失败时不会阻塞消息处理，只是丢失线程上下文

线程与会话 Key 的映射

线程上下文最终映射到 OpenClaw 的会话系统。线程内的对话使用独立的 Session Key：

```
// src/slack/monitor/message-handler/prepare.ts
const threadContext = resolveSlackThreadContext({ message, replyToMode: ctx.replyToMode
const isThreadReply = threadContext.isThreadReply;

const threadKeys = resolveThreadSessionKeys({
  baseSessionKey,
  threadId: isThreadReply ? threadTs : undefined,
  parentSessionKey: isThreadReply && ctx.threadInheritParent ? baseSessionKey : undefin
});
const sessionKey = threadKeys.sessionKey;
```

`threadHistoryScope` 配置决定线程回复的历史记录归属：

threadHistoryScope	行为
`"thread"` （默认）	线程内的回复使用独立的历史记录
`"channel"`	线程回复共享频道级历史记录

回复线程目标

`replyToMode` 配置控制 Bot 回复消息时是否自动创建/加入线程：

```
// src/slack/threading.ts
export function resolveSlackThreadTargets(params) {
  const { incomingThreadTs, messageTs } = resolveSlackThreadContext(params);
  const replyThreadTs = incomingThreadTs
    ?? (params.replyToMode === "all" ? messageTs : undefined);
  const statusThreadTs = replyThreadTs ?? messageTs;
  return { replyThreadTs, statusThreadTs };
}
```

replyToMode	效果
`"off"`	不自动开线程，直接在频道回复
`"first"`	仅第一条回复进入线程，后续消息不强制

replyToMode	效果
`"all"`	所有回复都在线程中

`"first"` 模式通过共享的 `hasRepliedRef` 引用实现：

```
// src/slack/monitor/message-handler/dispatch.ts
const hasRepliedRef = { value: false };
const replyPlan = createSlackReplyDeliveryPlan({
  replyToMode: ctx.replyToMode,
  incomingThreadTs,
  messageTs,
  hasRepliedRef,
});

// 在 deliver 回调中使用
deliver: async (payload) => {
  const replyThreadTs = replyPlan.nextThreadTs();
  await deliverReplies({ ..., replyThreadTs });
  replyPlan.markSent();  // 标记已发送第一条
}
```

线程 Starter 内容注入

当用户在线程内回复时，OpenClaw 会尝试获取线程根消息的内容，作为上下文注入到 Agent 的输入中：

```
if (isThreadReply && threadTs) {
  const starter = await resolveSlackThreadStarter({
    channelId: message.channel,
    threadTs,
    client: ctx.app.client,
  });
  if (starter?.text) {
    threadStarterBody = formatThreadStarterEnvelope({
      channel: "Slack",
      author: starterName,
      body: starterWithId,
      envelope: envelopeOptions,
    });
  }
  // 如果当前消息没有附件但线程根消息有，也会一并获取
  if (!media && starter.files && starter.files.length > 0) {
    threadStarterMedia = await resolveSlackMedia({ files: starter.files, ... });
  }
}
```

这确保了 Agent 即使只接收到线程中间的一条回复，也能理解整个对话的起始背景。

消息入站防抖

Slack 通道复用了 OpenClaw 通用的入站防抖机制（详见第 6 章），但 Slack 的防抖 Key 构建更加精细：

```
buildKey: (entry) => {
  const senderId = entry.message.user ?? entry.message.bot_id;
  const threadKey = entry.message.thread_ts
    ? `${entry.message.channel}:${entry.message.thread_ts}`
    : entry.message.parent_user_id && messageTs
      ? `${entry.message.channel}:maybe-thread:${messageTs}`
      : entry.message.channel;
  return `slack:${ctx.accountId}:${threadKey}:${senderId}`;
}
```

防抖 Key 包含四个维度：通道 ID + 账户 ID + 线程/频道 ID + 发送者 ID。这确保了：

- 不同频道的消息互不干扰
- 同一频道中不同线程的消息独立防抖

- 同一线程中不同用户的消息独立防抖

Markdown → mrkdwn 格式转换

Slack 使用自定义的 `mrkdwn` 格式（注意不是标准 Markdown），OpenClaw 需要做格式转换：

```
// src/slack/format.ts
export function markdownToSlackMrkdwn(markdown: string, options = {}): string {
  const ir = markdownToIR(markdown, {
    headingStyle: "bold",       // Slack 不支持 # 标题，用粗体替代
    blockquotePrefix: "> ",     // 引用格式一致
  });
  return renderMarkdownWithMarkers(ir, {
    styleMarkers: {
      bold: { open: "*", close: "*" },          // Slack: *bold*
      italic: { open: "_", close: "_" },        // Slack: _italic_
      strikethrough: { open: "~", close: "~" }, // Slack: ~strike~
      code: { open: "`", close: "`" },
      code_block: { open: "```\n", close: "```" },
    },
    escapeText: escapeSlackMrkdwnText,
    buildLink: buildSlackLink,  // [text](url) → <url|text>
  });
}
```

Slack mrkdwn 中 `<>` 有特殊含义（用于链接、@mention 等），因此需要对用户内容中的这些字符进行转义，同时保留 Slack 原生的尖括号标记（如 `<@U123456>`）：

```
function isAllowedSlackAngleToken(token: string): boolean {
  const inner = token.slice(1, -1);
  return (
    inner.startsWith("@") ||     // @mention
    inner.startsWith("#") ||     // #channel
    inner.startsWith("!") ||     // !special commands
    inner.startsWith("http://")  // 链接
    // ...
  );
}
```

斜杠命令系统

Slack 斜杠命令（Slash Commands）是 Bot 与用户交互的另一种重要方式。OpenClaw 支持两种模式：

1. **统一命令**：所有输入通过一个配置好的命令名（如 `/openclaw`）接收
2. **原生命令**：将 OpenClaw 的内置命令（如 `/reset`、`/model` 等）注册为独立的 Slack 斜杠命令

```
// src/slack/monitor/slash.ts
if (nativeCommands.length > 0) {
  for (const command of nativeCommands) {
    ctx.app.command(`/${command.name}`, async ({ command: cmd, ack, respond }) => {
      // 解析命令参数并分发
    });
  }
} else if (slashCommand.enabled) {
  // 使用统一的命令名
  ctx.app.command(
    buildSlackSlashCommandMatcher(slashCommand.name),
    async ({ command, ack, respond }) => {
      await handleSlashCommand({ command, ack, respond, prompt: command.text });
    }
  );
}
```

斜杠命令使用 `ephemeral` 响应方式（仅命令发起者可见），且拥有独立的 Session Key：

```
SessionKey: `agent:${route.agentId}:${slashCommand.sessionPrefix}:${command.user_id}`
```

此外，OpenClaw 还支持交互式参数菜单——当命令有多个可选参数时，Bot 会通过 Slack Block Kit 的按钮菜单让用户选择，而非要求用户手动输入参数文本。

本节小结

1. **Slack 通道是 OpenClaw 中最复杂的通道实现**，包含 65 个源文件，覆盖消息、反应、成员变动、频道事件和斜杠命令等全方位集成。
2. **Bolt SDK 提供了事件处理的基础框架**，OpenClaw 在其上构建了 Socket Mode 和 HTTP Mode 两种连接方式，通过 `monitorSlackProvider()` 统一编排。

3. **双 Token 架构中**，Bot Token（`xoxb-`）用于 API 调用，App Token（`xapp-`）用于 Socket Mode 连接；OpenClaw 支持从配置文件和环境变量两个来源解析 Token，并进行 `api_app_id` 一致性校验。
4. **线程会话管理是 Slack 通道最精巧的部分**，包括 `thread_ts` 缺失恢复、线程级 Session Key 隔离、`replyToMode` 三档控制（off/first/all）、线程 Starter 内容注入等机制。
5. **消息格式转换方面**，OpenClaw 通过 Markdown IR 中间表示将标准 Markdown 转换为 Slack mrkdwn 格式，并精确处理尖括号标记的转义。
6. **斜杠命令系统支持统一命令和原生命令两种模式**，并能通过 Block Kit 按钮提供交互式参数选择体验。